SRA-RiskGate: A Benchmark and a 4B Model for Gating Stablecoin Payments Made by AI Agents, and the Checks the Model Still Needs

Sriram Ramakrishnan
Independent Researcher
Sriram.Ramakrishnan@ieee.org

October 2026

Abstract

AI agents increasingly pay for services in stablecoins, through protocols such as x402 and EIP-3009 authorizations. These payments are final once settled, so a wrong decision before payment cannot be undone by a chargeback, and a wrong decision in a later dispute cannot be corrected by reversing the transfer. We release three artifacts. *SRA Stablecoin Risk Bench* is a synthetic benchmark of 20,000 cases with real secp256k1 signatures and rule-engine labels, covering a pre-payment risk gate (approve, hold or reject) and post-payment dispute adjudication across the settlement-finality boundary. *SRA-RiskGate-4B* is a QLoRA fine-tune of Qwen3-4B-Instruct-2507 that raises payment decision accuracy on the held-out test split from 59.8% to 99.5% and lowers unsafe approvals from 34.6% to 0.47%. *sra-riskgate* is an open-source library of deterministic checks that sit around the model. We then analyse where the model still fails. All four unsafe approvals are prompt injections whose payment text vouches for itself; a deterministic claims check holds all four. In disputes, 26.0% of valid refunds are wrongly denied, and we trace 59 of the 61 cases to one operation: deciding whether the claim was filed inside the policy's window, where the model's "outside the window" claims are wrong 81% of the time. A short deterministic date check lowers wrongful denials to 0.85% without adding wrongful refunds. A contamination analysis finds no verbatim train/test overlap and 99.3% accuracy on test cases with no structural twin in training. All predictions, scripts and models are public.

1 Introduction

Payment protocols designed for software agents let an agent pay for an API call or a dataset by signing a stablecoin authorization in the middle of an HTTP exchange [13, 26]. Two properties make this setting unusual for machine-learning safety. First, settlement is final: once a USDC transfer confirms, no intermediary can claw it back, so the only protection is a decision made *before* signing. Second, the payment request itself is attacker-controlled text that reaches the agent's model, which makes indirect prompt injection [5, 11, 27] a direct path to moving money.

We study a narrow but consequential task: given a payment request, the receiving policy and the results of deterministic verification tools (signature recovery, risk attestations, sanctions screening), decide whether to `approve`, `hold` or `reject`; and, after payment, given a dispute and its evidence, choose a remedy that is actually possible given where the money is. Our contributions are:

1. A public benchmark of 20,000 synthetic cases (18,000 train, 2,000 test) with an open scorer and safety-oriented metrics (Section 3).
2. A 4B-parameter model that runs on a laptop and reaches 99.5% payment decision accuracy with 0.47% unsafe approvals, against 59.8% and 34.6% for its base model (Section 5).

3. An error analysis showing that the remaining failures concentrate in two operations that are better done in code: trusting self-asserted claims in untrusted text, and comparing dates. For each we release a deterministic check and measure its effect on the published predictions (Section 6).
4. A contamination analysis for template-generated benchmarks (Section 7).

The design principle that emerges is simple: *the model judges, deterministic checks enforce.* The model reads messy, partly adversarial evidence; arithmetic, signature recovery, list membership and claim detection belong in code that cannot be talked out of its answer.

2 Background

Agent payments. x402 [26] revives the HTTP 402 status code: a server answers a request with payment requirements (network, asset, amount, recipient), and the client retries with a signed EIP-3009 `transferWithAuthorization` [13] that a facilitator submits on chain. Agents can also send ordinary ERC-20 `transfer` transactions. Both are push payments: the payer's key authorises the movement of funds, and confirmation makes it final.

Finality and remedies. We follow the framework of [17], which separates four terms that card payments blur together. A *reversal* undoes a held payment before release and needs a smart contract with that power (escrow, a 2-of-3 multisig). A *refund* after finality is a new, voluntary forward transfer from the merchant. A *chargeback*, a forced extraction by an intermediary, does not exist on these rails. A *dispute* is the adjudication that resolves into one of the others. After finality, an arbiter can force compensation only from collateral the merchant posted in advance (a bond), and the payout is a compensating transaction in the sense of the Saga pattern [10]: it must be idempotent, funded and correctly addressed.

Risk attestations. The benchmark uses a small EIP-712 [2] signed statement ("attester A screened address S on chain C at time T and rated it L, expiring at E") that travels with the payment request. Attestations are advisory: the verifier's own sanctions screening always overrides a clean attestation.

3 The SRA Stablecoin Risk Bench

Tasks. Each case gives the model the current time, the policy, the payload (the payment or dispute exactly as received, marked untrusted) and the results of verification tools. *Risk gate* cases cover three payment formats (x402 requirements, EIP-3009 authorizations, ERC-20 transfers) and ask for a JSON verdict with a decision, a risk level, reason codes (*flags*) and required actions. *Dispute* cases cover three rails (direct payment, escrow, 2-of-3 multisig) and ask for an outcome, a mechanism, the settlement state, and, for refunds, the exact amount, destination and idempotency key. There are ten dispute outcomes and eight mechanisms; there is deliberately no "chargeback" outcome.

Generation and labels. Cases are produced by a deterministic generator that applies perturbations to clean cases: forged, expired, untrusted or out-of-scope attestations; sanctioned parties; expired or not-yet-valid authorizations; amounts over policy limits; wrong assets or networks; recipient and amount mismatches; and prompt-injection text hidden in untrusted fields. All signatures are real secp256k1 EIP-3009 and EIP-712 signatures over simulated keys. Every label is computed by a rule engine whose decision order is documented in the dataset's specification; labels never

propose a remedy the settlement state cannot support. The generator itself is not yet public, which we list as a limitation.

Splits. The data has 20,000 cases: 18,000 for training and 2,000 held out. The test split contains 1,175 risk-gate cases (847 that should be held or rejected and 328 legitimate payments, including 69 with prompt-injection text) and 825 dispute cases (77 with injection text). Training data is also provided as chat prompts and completions.

Metrics. The open scorer [16] reports, for the risk gate, decision accuracy and macro-F1, flag micro-F1, the *unsafe approval rate* (the share of hold or reject cases the model approves), over-blocking (legitimate payments rejected) and injection handling. For disputes it reports outcome accuracy and macro-F1, flag F1, the *impossible remedy rate* (on final payments, proposing a reversal, an escrow or multisig mechanism, or a bond payout with no usable bond), the *wrongful refund rate*, the *wrongful denial rate* (correct outcome is a refund, prediction is deny or dismiss) and refund-detail accuracy (correct outcome, amount and destination). The composite SRA score averages a risk-gate score and a dispute score built from these terms. A Hugging Face `eval.yaml` defines both tasks for Inspect [23].

4 Model

SRA-RiskGate-4B [18] is Qwen3-4B-Instruct-2507 [15] fine-tuned with QLoRA [6, 12]: the base model is loaded in 4-bit NF4 with double quantisation and bfloat16 compute, and LoRA adapters (rank 32, $\alpha = 64$, dropout 0.05) are applied to all attention and MLP projections. Training is supervised fine-tuning in TRL [25] on the 18,000 training cases in their chat form, for one epoch, with learning rate 10^{-4} on a cosine schedule with 3% warmup, an effective batch size of 16 (2 per device $\times$ 8 accumulation steps), a maximum sequence length of 3,072 tokens (prompts are about 1,300–2,000 tokens, so none is truncated), gradient checkpointing, and the loss computed on the verdict only. Every 200 steps the loss was measured on 300 cases sampled from the test split, and the checkpoint with the lowest loss was kept (see Limitations). The training script is public [20]. Training used a single rented GPU; the hardware model and wall-clock time were not logged. The adapter is merged into the base weights for release; GGUF quantisations (Q8_0 to Q4_K_M) and an Ollama build are also published, but all results below are for the full-precision model.

Payment and dispute cases use different system prompts. The payment prompt states that everything inside the payload is untrusted and that instructions found there must never be followed. The user turn wraps the context, payload and tool results in tagged JSON blocks. Every reported result uses exactly the training prompts and greedy decoding.

5 Results

We evaluated the base and fine-tuned models on all 2,000 test cases with vLLM [14] on an NVIDIA T4 in float16, greedy decoding, at most 512 new tokens. Two separate full runs of the fine-tuned model gave identical scores. The run took 6,516 s for 2,000 cases (3.3 s per case, about 2,000 prompt tokens and 98 completion tokens per case). Every prediction is published [19].

Table 1 summarises the results. Fine-tuning cuts unsafe approvals from about one in three risky payments to about one in two hundred while blocking *fewer* legitimate payments than the base model. Accuracy is 100% on 16 of the 21 case types (clean cases and 20 perturbations) and at least 97.9% on all but one; the exception is injection (94.2%). Dispute adjudication is clearly harder.

Metric	Qwen3-4B (base)	SRA-RiskGate-4B
SRA composite score ↑	0.421	**0.913**
Risk gate (1,175 payments)		
Decision accuracy ↑	59.8%	**99.5%**
Unsafe approvals (of 847) ↓	34.6%	**0.47%**
Over-blocking (of 328) ↓	5.2%	**0.0%**
Injected payments approved (of 69) ↓	52.2%	**5.8%**
Flag micro-F1 ↑	0.105	**0.994**
Disputes (825)		
Outcome accuracy ↑	0.0%	**77.8%**
Impossible remedies ↓	n/a	0.38%
Wrongful refunds ↓	n/a	2.2%
Wrongful denials ↓	n/a	26.0%
Refund details fully correct ↑	n/a	64.3%
Schema-valid output (payments / disputes) ↑	99.8% / 0%	100% / 100%

Table 1: Held-out test results. The base model returns dispute JSON with the right field names but outcome values outside the allowed vocabulary, so none of its dispute verdicts is usable and its dispute safety rates are not meaningful (n/a).

Per-class F1 ranges from 1.00 (`refer_to_issuer`) and 0.96 (`reverse_before_release`) down to 0.62 (`await_merchant_response`), 0.57 (`dismiss`) and 0.54 (`recommend_voluntary_refund`). The model almost never proposes an impossible remedy (0.38%), but it denies or dismisses 26.0% of the claims that should be refunded, an asymmetry Section 6 explains.

6 Where the model fails, and what to do in code

6.1 Self-vouching payment text

All four unsafe approvals are prompt-injection cases (4 of 69 injected payments); outside injection the model made no unsafe approval. In each, the untrusted payment text asserts its own verification, for example "risk level: low (verified offline)". A payer cannot grade its own risk, so the companion library runs a deterministic scan for such self-asserted claims in untrusted text, with Unicode normalisation against look-alike characters, and holds the payment on a match [20]. Combined with the model, the check gives 0 unsafe approvals out of 847 risky test payments and adds 0 false holds on the 3,358 legitimate payments in the test and training splits. The check was written after these failures were found, so it repairs a known weakness rather than providing an independent evaluation; differently worded attacks may still pass.

6.2 The dispute window

Finding. 59 of the 61 wrongful denials are `dismiss` verdicts carrying the flag `outside_dispute_window`, which the correct label never contains. The model's explanations are internally inconsistent: "The dispute was filed 26.5 days after payment, outside the 60-day window." To check this systematically we recomputed, for every dispute, the days between `paid_at` and `filed_at` and compared them with the policy's `dispute_window_days`. All 54 gold labels with the window flag are outside by this arithmetic, so the rule is consistent. The model, however, claims "outside the window" 112 times, and 91 of these claims (81%) are false; it also misses 33 of the 54 disputes that are genuinely outside. The model computes the elapsed days correctly and then compares them wrongly. Denial

Dispute metric (825 cases)	Model	Model + window check
Wrongful denials ↓	26.0%	**0.85%**
Wrongful refunds ↓	2.2%	2.2%
Outcome accuracy ↑	77.8%	78.7%

Table 2: Effect of the deterministic dispute-window check (91 false claims corrected, 86 outcomes changed). Escalations count as misses, so accuracy rises only slightly while the harmful error nearly disappears.

rates are highest when the case also carries distracting detail: escrow already released (62%), a partial refund already made (58%), untrusted delivery evidence (6 of 7).

Check. Whether a dispute is inside its window is date arithmetic, so we move it to code. `check_dispute_verdict` [20] recomputes the window from the dispute's own dates. If the verdict claims the dispute is outside but the dates put it inside, the false flag is replaced, and a `deny` or `dismiss` resting on it becomes `escalate_to_human` with no refund. The check never produces a refund. Table 2 shows its effect on the published predictions.

6.3 Raw transactions and signatures

Agents sign transactions, not descriptions of payments. The library's `check_transaction` decodes the exact transaction about to be signed: a plain USDC `transfer` is passed to the full payment check, and token approvals (including unlimited `approve`), `transferFrom`, calls to other contracts and native-token transfers are held, because an allowance lets another address move the wallet's funds later without any further check. The same rule is enforced at the wallet in the ElizaOS plugin, which wraps a viem wallet client so that only approved USDC transfers are broadcast; a batch of calls (EIP-5792 `sendCalls` [8]) is approved only if every call is an approved USDC transfer, with payments to the same payee summed so that splitting a payment does not bring it under the amount limit.

In x402 the agent signs typed data instead of a transaction, and anyone holding the signature can move the funds until it expires. `check_authorization` therefore checks the EIP-712 payload itself before signing: the domain must be USDC on that chain, `from` must be the agent's wallet, the type definition must hash to the EIP-3009 one, and the authorization must be current and short-lived (by default it is held if valid for more than an hour); an EIP-2612 `Permit` [7] is held as an approval. Two signing paths need their own rules. An EIP-7702 [9] authorization delegates the account to contract code, on every chain if signed with chain id 0, so any transaction carrying an authorization list, and any standalone authorization, is held. Circle Gateway's batched payments [4] use the same `TransferWithAuthorization` under Gateway's own domain and must stay valid for at least three days; they are accepted only with Circle's published GatewayWallet contract as the verifying contract, under a separate validity limit. Our first version of that limit (seven days) held every payment produced by Circle's own client SDK, which signs for seven days plus 100 s; we found this by testing against payments generated with the SDK, and such a payment is now part of the test suite. On Arc [3], where USDC is the native asset with 18 decimals and also has a 6-decimal ERC-20 interface, a plain native send is checked as a USDC payment, rounded up to a whole micro-USDC so that no limit can be passed by dust.

Three further paths need checks of their own. With custodial developer-controlled wallets (Circle Wallets), the agent does not sign at all: it sends an API request, and the provider signs as soon as it

accepts it. `guard_circle_wallets` therefore wraps the provider's client so that transfers, contract executions and typed-data or transaction signing requests are checked before they are sent, and a request that is not approved never reaches the provider. Address poisoning [22] plants addresses that share the first and last characters of a payee the victim already uses, in the hope that the victim copies the wrong one; a recipient that is not a known payee but matches one in at least seven hexadecimal characters, at least three at each end, is rejected (a random address matches by chance about once in 1.4×10^8). Finally, an account that has already been delegated through EIP-7702 runs the delegate's code, which can move funds without any further signature; when the caller supplies the sender's account code (from `eth_getCode`), a delegation designator pointing to a contract not on an expected list holds every payment and tells the agent to stop signing. The ElizaOS plugin reads the account code itself before every guarded call.

7 Contamination analysis

Template-generated benchmarks risk test cases that differ from training cases only in surface values [21]. No test input appears verbatim in the training split. To detect structural overlap, we masked every hexadecimal value and every number in each input and compared the results. 627 of the 1,175 payment test cases have such a "template twin" in training (531 of them with the same label); no dispute case does. Accuracy barely depends on it: 99.7% on payment cases with a twin and 99.3% on the 548 without one. All four unsafe approvals fall in the no-twin group, so on cases unlike anything in training the unsafe-approval rate is 0.83% (4 of 481) rather than 0.47%. We report both.

8 Release and deployment

The benchmark (CC-BY-4.0), the model and its LoRA adapter (Apache-2.0), all predictions, the scorer and the analysis scripts are public. The library `sra-riskgate` provides the checks above together with an offline match against OFAC's published sanctioned digital-currency addresses [24] (a dated snapshot, not a screening service), known-payee and blocklist policies, and depeg limits. It is exposed to agents through an MCP server [1] with eight read-only tools, LangChain and CrewAI integrations, a Coinbase AgentKit action provider, a GitHub Action that checks agent-generated calldata in CI, and the ElizaOS plugin, which we ran inside a live ElizaOS 1.7.2 agent on a test network: it blocked a payment to a blocklisted address, held a self-vouching request and a payment to a new payee, and approved a clean payment to a known payee (checked only; no funds were sent). Served with vLLM on a single 24 GB GPU, the model answered individual checks in 1.2–1.6 s from a remote client.

We also ran the checks on Arc's public test network with test USDC. A demo agent proposed four payments, each checked before signing: a 0.10 USDC ERC-20 transfer and a 0.05 USDC native send to a known payee were approved, signed and confirmed; an unlimited approval and a self-vouching request were held and never signed. The payee's balance of exactly 0.15 USDC confirmed both decimal scales on chain. In a second run the agent deposited 0.10 USDC into Circle Gateway and proposed four Gateway payments: one, built as Circle's SDK builds it, was approved, signed and accepted by Gateway's batch API, and three were held before signing (a look-alike verifying contract one character off Circle's, a signature valid for 30 days, and a payee not on the known list). In a third run the agent was given only guarded clients for a Circle developer-controlled wallet and proposed four requests: a 0.10 USDC transfer to the known payee was approved, sent to Circle and completed on chain, while an unlimited `approve` (a contract execution), a payment to an unknown

address and an unlimited EIP-2612 permit (a typed-data signing request) were stopped and never sent to Circle.

9 Limitations

- **Synthetic data.** Scores measure agreement with the generator's rule engine, not accuracy on real payment traffic; the policy space is narrow and a small set of attester addresses recurs. A real-data test set is the most important next step.
- **Generator not public.** Labels can be re-scored but not regenerated.
- **Small slices.** Some categories are small (69 injected payments, 7 cases with untrusted delivery evidence), so their rates carry wide uncertainty.
- **Post-hoc checks.** Both deterministic checks were designed after inspecting the test errors they fix. Their measured effect is an upper bound on what to expect for new attack wordings or new error types.
- **Single author, single base model.** The only comparison is the base model; larger general-purpose models and third-party reproduction are still missing.
- **Checkpoint selection used test cases.** The training script kept the checkpoint with the lowest loss on 300 cases sampled from the test split, rather than on a separate validation split. The selection used loss, not the reported metrics, over a handful of checkpoints from a single epoch, so we expect the effect to be small, but the reported results may be slightly optimistic. Version 2 will select on a validation split carved from the training data.
- **Quantised builds** have not been evaluated.
- **On-chain runs are demonstrations.** The Arc test-network runs cover twelve proposed payments, chosen by us; they show the checks working against live contracts and Circle's API, not their error rates on real traffic.

10 Conclusion

A small fine-tuned model can gate agent stablecoin payments with high accuracy and few unsafe approvals, but its remaining errors are not random. They cluster in operations a model should not be trusted with when money moves irreversibly: believing what untrusted text says about itself, and comparing two numbers. Moving those operations to deterministic code removes almost all of the measured harm. We release the benchmark, the model, the predictions and the checks so that others can test this design on their own policies.

Use of AI assistance. Parts of the software and of this manuscript were drafted with the help of an AI assistant (Claude); all results were run and checked by the author, who takes responsibility for the content.

References

[1] Anthropic. Model context protocol. `https://modelcontextprotocol.io`, 2024.

[2] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed structured data hashing and signing. Ethereum Improvement Proposals, no. 712, 2017. URL `https://eips.ethereum.org/EIPS/eip-712`.

[3] Circle. Arc documentation. Developer documentation, 2026. URL https://docs.arc.io.

[4] Circle. Circle Gateway: Nanopayments and EIP-3009 signing. Developer documentation, 2026. URL https://developers.circle.com/gateway/nanopayments.

[5] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024.

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.

[7] EIP-2612. EIP-2612: Permit extension for EIP-20 signed approvals. Ethereum Improvement Proposals, no. 2612, 2020. URL https://eips.ethereum.org/EIPS/eip-2612.

[8] EIP-5792. EIP-5792: Wallet call API. Ethereum Improvement Proposals, no. 5792, 2022. URL https://eips.ethereum.org/EIPS/eip-5792.

[9] EIP-7702. EIP-7702: Set EOA account code. Ethereum Improvement Proposals, no. 7702, 2024. URL https://eips.ethereum.org/EIPS/eip-7702.

[10] Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987.

[11] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 2023.

[12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[13] Peter Jihoon Kim, Kevin Britz, and David Knott. EIP-3009: Transfer with authorization. Ethereum Improvement Proposals, no. 3009, 2020. URL https://eips.ethereum.org/EIPS/eip-3009.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[15] Qwen Team. Qwen3 technical report. arXiv:2505.09388, 2025.

[16] Sriram Ramakrishnan. SRA stablecoin risk bench. https://huggingface.co/datasets/sriram1983007/sra-stablecoin-risk-bench, 2026.

[17] Sriram Ramakrishnan. Refunds and reversals when stablecoin payments are final. BLOG@CACM, Communications of the ACM, 2026.

[18] Sriram Ramakrishnan. SRA-RiskGate-4B. https://huggingface.co/sriram1983007/SRA-RiskGate-4B, 2026. doi:10.57967/hf/11215.

[19] Sriram Ramakrishnan. SRA benchmark results: published predictions. https://huggingface.co/datasets/sriram1983007/sra-bench-results, 2026.

[20] Sriram Ramakrishnan. sra-riskgate: deterministic checks for stablecoin payments made by AI agents. https://github.com/sriram1983007-dev/sra-riskgate, 2026. Version 0.13.1.

[21] Oscar Sainz, Jon Ander Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.

[22] Taro Tsuchiya, Jin-Dong Dong, Kyle Soska, and Nicolas Christin. Blockchain address poisoning. In *Proceedings of the 34th USENIX Security Symposium*, 2025.

[23] UK AI Security Institute. Inspect: A framework for large language model evaluations. https://github.com/UKGovernmentBEIS/inspect_ai, 2024.

[24] U.S. Department of the Treasury, Office of Foreign Assets Control. Specially designated nationals and blocked persons list. https://ofac.treasury.gov. Digital currency addresses as extracted by https://github.com/0xB10C/ofac-sanctioned-digital-currency-addresses.

[25] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer reinforcement learning. https://github.com/huggingface/trl, 2020.

[26] x402 Foundation. x402: An open standard for internet-native payments. https://github.com/x402-foundation/x402, 2025. Originally released by Coinbase.

[27] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.